

```

# =====

# SAC PRACTICAL - STUDENT MARKS DATASET

# =====

# =====

# QUESTION 1

# Create vectors storing register number, name, marks

# =====

# Register Numbers

regno <- c(101,102,103,104,105,106,
          107,108,109,110,111,112)

# Student Names

name <- c("Arun","Bala","Cathy","David",
          "Esha","Farhan","Gokul","Hari",
          "Isha","John","Kiran","Latha")

# Student Marks

marks <- c(78,85,67,90,88,76,
          95,72,81,69,84,91)

# Display marks

print(marks)

# =====

```

```
# Compute total and average marks
```

```
# =====
```

```
# Total Marks
```

```
total_marks <- sum(marks)
```

```
# Average Marks
```

```
average_marks <- mean(marks)
```

```
# Display results
```

```
print(paste("Total Marks =", total_marks))
```

```
print(paste("Average Marks =", average_marks))
```

```
# =====
```

```
# Identify maximum and minimum marks
```

```
# =====
```

```
# Maximum mark
```

```
max_marks <- max(marks)
```

```
# Student with maximum mark
```

```
topper <- name[which.max(marks)]
```

```
print(paste("Maximum Marks =", max_marks))
```

```
print(paste("Topper =", topper))
```

```
# Minimum mark
```

```
min_marks <- min(marks)
```

```
# Student with minimum mark
```

```
low_student <- name[which.min(marks)]
```

```
print(paste("Minimum Marks =", min_marks))
```

```
print(paste("Lowest Scorer =", low_student))
```

```
# =====
```

```
# Compare average marks of first 6 and next 6 students
```

```
# =====
```

```
# Average of first 6 students
```

```
avg_first6 <- mean(marks[1:6])
```

```
# Average of next 6 students
```

```
avg_next6 <- mean(marks[7:12])
```

```
# Display averages
```

```
print(paste("Average of First 6 Students =", avg_first6))
```

```
print(paste("Average of Next 6 Students =", avg_next6))
```

```
# =====  
# Convert vectors into data frame  
# =====
```

```
student_df <- data.frame(regno, name, marks)
```

```
# Display data frame  
print(student_df)
```

```
# Display structure  
str(student_df)
```

```
# =====  
# QUESTION 2  
# Import dataset and preprocessing  
# =====
```

```
# NOTE:  
# Create a CSV file named student.csv  
# and upload it in online R compiler
```

```
# Import dataset  
df <- read.csv("student.csv")
```

```
# Display dataset
```

```
print(df)
```

```
# =====
```

```
# Handle Missing Values
```

```
# =====
```

```
# Check missing values
```

```
print(is.na(df))
```

```
# Replace missing marks with average marks
```

```
df$marks[is.na(df$marks)] <- mean(df$marks,  
                                na.rm = TRUE)
```

```
# Display updated dataset
```

```
print(df)
```

```
# =====
```

```
# Handle Invalid Values
```

```
# Example:
```

```
# Remove marks less than 0 or greater than 100
```

```
# =====
```

```
df <- df[df$marks >= 0 & df$marks <= 100, ]
```

```
print(df)
```

```
# =====
```

```
# Normalize the marks
```

```
# =====
```

```
df$normalized_marks <- scale(df$marks)
```

```
print(df)
```

```
# =====
```

```
# Descriptive Statistical Analysis
```

```
# =====
```

```
# Summary statistics
```

```
summary(df)
```

```
# Mean
```

```
mean_marks <- mean(df$marks)
```

```
# Median
```

```
median_marks <- median(df$marks)
```

```
# Standard Deviation
```

```
sd_marks <- sd(df$marks)
```

```
# Variance
```

```
variance_marks <- var(df$marks)
```

```
# Display results
```

```
print(paste("Mean =", mean_marks))
```

```
print(paste("Median =", median_marks))
```

```
print(paste("Standard Deviation =", sd_marks))
```

```
print(paste("Variance =", variance_marks))
```

```
# =====
```

```
# Visualize distribution of marks
```

```
# =====
```

```
hist(df$marks,
```

```
  main = "Distribution of Student Marks",
```

```
  xlab = "Marks",
```

```
  col = "skyblue",
```

```
  border = "black")
```

```
# =====
```

```
# Statistical Test
```

```
# Check whether average marks differ from 70
```

```
# =====
```

```
# One Sample t-test
```

```
test_result <- t.test(df$marks, mu = 70)
```

```
# Display test result
```

```
print(test_result)
```

```
# =====
```

```
# QUESTION 3
```

```
# MACHINE LEARNING MODEL
```

```
# =====
```

```
# =====
```

```
# Split dataset into training and testing sets
```

```
# =====
```

```
# Set seed for reproducibility
```

```
set.seed(100)
```

```
# Generate training index
```

```
index <- sample(1:nrow(df),
```

```
0.7 * nrow(df))
```

```
# Training data
```



```
train <- df[index, ]
```

```
# Testing data
```

```
test <- df[-index, ]
```

```
# Display train and test data
```

```
print(train)
```

```
print(test)
```

```
# =====
```

```
# Apply Decision Tree Algorithm
```

```
# =====
```

```
# Install package if not available
```

```
# install.packages("rpart")
```

```
# Load library
```

```
library(rpart)
```

```
# NOTE:
```

```
# Dataset should contain a target column named "result"
```

```
# Example values:
```

```
# Pass / Fail
```

```
# Build Decision Tree Model
```

```
model <- rpart(result ~ marks,
```

```

        data = train,
        method = "class")

# Display model
print(model)


# =====

# Prediction using test data

# =====

pred <- predict(model,
                test,
                type = "class")

print(pred)


# =====

# Confusion Matrix

# =====

confusion_matrix <- table(pred,
                          test$result)

print(confusion_matrix)

```

```
# =====  
# Accuracy Calculation  
# =====
```

```
accuracy <- mean(pred == test$result)
```

```
print(paste("Accuracy =", accuracy))
```

```
# =====  
# Visualize Decision Tree  
# =====
```

```
plot(model)
```

```
text(model)
```

```
# =====  
# END OF PROGRAM  
# =====
```

```
# =====
```

```
# SAC PRACTICAL - HOUSING DATASET
```

```
# REAL ESTATE COMPANY
```

```
# =====
```

```
# =====
```

```
# QUESTION 1
```

```
# Create matrix representing price, size and rooms
```

```
# =====
```

```
housing_matrix <- matrix(c(
```

```
  # Price
```

```
  5000000, 6500000, 7200000, 8000000,
```

```
  # Size
```

```
  1200, 1500, 1800, 2000,
```

```
  # Rooms
```

```
  2, 3, 3, 4
```

```
),
```

```
nrow = 4,
```

```
ncol = 3,
```

```
byrow = FALSE)
```

```
# Column names
```

```
colnames(housing_matrix) <- c("Price",  
                              "Size",  
                              "Rooms")
```

```
# Row names
```

```
rownames(housing_matrix) <- c("House1",  
                              "House2",  
                              "House3",  
                              "House4")
```

```
# Display matrix
```

```
print(housing_matrix)
```

```
# =====
```

```
# Compute Row-wise Totals
```

```
# =====
```

```
row_totals <- rowSums(housing_matrix)
```

```
print("Row Wise Totals")
```

```
print(row_totals)
```

```
# =====
```

```
# Compute Column-wise Totals
```

```
# =====
```

```
column_totals <- colSums(housing_matrix)
```

```
print("Column Wise Totals")
```

```
print(column_totals)
```

```
# =====
```

```
# Identify House with Maximum Price
```

```
# =====
```

```
max_price <- max(housing_matrix[, "Price"])
```

```
max_house <- rownames(housing_matrix)[which.max(  
    housing_matrix[, "Price"])]
```

```
print(paste("Maximum Price =", max_price))
```

```
print(paste("House with Maximum Price =", max_house))
```

```
# =====
```

```
# Compare Features Across Houses
```

```
# =====
```

```
print("Comparison of House Features")
```

```
print(housing_matrix)
```

```
# =====
```

```
# Convert Matrix into Data Frame
```

```
# =====
```

```
housing_df <- as.data.frame(housing_matrix)
```

```
print(housing_df)
```

```
str(housing_df)
```

```
# =====
```

```
# QUESTION 2
```

```
# DATA PREPROCESSING
```

```
# =====
```

```
# Create Housing Dataset Directly
```

```
df <- data.frame(
```

```
Price = c(5000000,6500000,7200000,  
          8000000,7500000,6200000,  
          6800000,7900000),
```

```
Size = c(1200,1500,1800,  
         2000,1700,1400,  
         1600,1900),
```

```
Rooms = c(2,3,3,4,3,2,3,4),
```

```
Location = c("Urban","Urban",  
              "Suburban","Urban",  
              "Suburban","Urban",  
              "Suburban","Urban")
```

```
)
```

```
# Display dataset
```

```
print(df)
```

```
# =====
```

```
# Handle Missing Values
```

```
# =====
```

```
print(is.na(df))
```

```
# Replace missing values if any
```



```
df$Price[is.na(df$Price)] <- mean(df$Price,  
                                  na.rm = TRUE)
```

```
print(df)
```

```
# =====
```

```
# Handle Invalid Values
```

```
# Remove invalid prices
```

```
# =====
```

```
df <- df[df$Price > 0, ]
```

```
print(df)
```

```
# =====
```

```
# Standardize Variables
```

```
# =====
```

```
df$Price_Standardized <- scale(df$Price)
```

```
df$Size_Standardized <- scale(df$Size)
```

```
print(df)
```

```
# =====  
  
# Statistical Analysis  
  
# =====  
  
summary(df)  
  
mean_price <- mean(df$Price)  
  
median_price <- median(df$Price)  
  
sd_price <- sd(df$Price)  
  
variance_price <- var(df$Price)  
  
print(paste("Mean Price =", mean_price))  
  
print(paste("Median Price =", median_price))  
  
print(paste("Standard Deviation =", sd_price))  
  
print(paste("Variance =", variance_price))  
  
  
  
# =====  
  
# Visualize Relationships
```

```
# =====
```

```
# NOTE:
```

```
# Commented because some online compilers
```

```
# do not support graphics
```

```
# plot(df$Size,
```

```
#   df$Price,
```

```
#   main = "Size vs Price",
```

```
#   xlab = "Size",
```

```
#   ylab = "Price")
```

```
# =====
```

```
# Statistical Test
```

```
# Compare prices between locations
```

```
# =====
```

```
urban_price <- df$Price[df$Location == "Urban"]
```

```
suburban_price <- df$Price[df$Location == "Suburban"]
```

```
test_result <- t.test(urban_price,  
                      suburban_price)
```

```
print(test_result)
```

```
# =====  
# QUESTION 3  
# MACHINE LEARNING MODEL  
# =====
```

```
# =====  
# Split Dataset into Training and Testing Sets  
# =====
```

```
set.seed(100)
```

```
index <- sample(1:nrow(df),  
               0.7 * nrow(df))
```

```
train <- df[index, ]
```

```
test <- df[-index, ]
```

```
print(train)
```

```
print(test)
```

```
# =====
```

```
# Apply Linear Regression
```

```
# =====
```

```
model <- lm(Price ~ Size + Rooms,  
            data = train)
```

```
print(summary(model))
```

```
# =====
```

```
# Prediction
```

```
# =====
```

```
pred <- predict(model,  
                test)
```

```
print(pred)
```

```
# =====
```

```
# RMSE Calculation
```

```
# =====
```

```
rmse <- sqrt(mean((pred - test$Price)^2))
```

```
print(paste("RMSE =", rmse))
```

```
# =====
```

```
# R Squared Calculation
```

```
# =====
```

```
r_squared <- summary(model)$r.squared
```

```
print(paste("R Squared =", r_squared))
```

```
# =====
```

```
# Visualize Results
```

```
# =====
```

```
# NOTE:
```

```
# Commented because online compilers
```

```
# may not support graphics
```

```
# plot(test$Price,
```

```
#   pred,
```

```
#   main = "Actual vs Predicted",
```

```
#   xlab = "Actual Price",
```

```
#   ylab = "Predicted Price")
```

```
# =====
```

```
# END OF PROGRAM
```

```
# =====
```

```
# =====
```

```
# SAC PRACTICAL - CUSTOMER DATASET
```

```
# COMPANY CUSTOMER INFORMATION
```

```
# =====
```

```
# =====
```

```
# QUESTION 1
```

```
# Create data frame with age, income and spending score
```

```
# =====
```

```
customer_df <- data.frame(
```

```
  CustomerID = c(1,2,3,4,5,6,7,8),
```

```
  Age = c(22,25,30,35,40,28,32,45),
```

```
  Income = c(25000,30000,45000,50000,  
            65000,40000,48000,70000),
```

```
  SpendingScore = c(80,75,60,55,
```

```
40,70,65,35),
```

```
Gender = c("Male","Female","Female",  
           "Male","Male","Female",  
           "Female","Male")  
)
```

```
# Display data frame  
print(customer_df)
```

```
# =====
```

```
# Convert Gender into Factor
```

```
# =====
```

```
customer_df$Gender <- as.factor(customer_df$Gender)
```

```
print(customer_df)
```

```
# =====
```

```
# Compute Summary Statistics
```

```
# =====
```

```
summary(customer_df)
```



```
mean_income <- mean(customer_df$Income)
```

```
mean_spending <- mean(customer_df$SpendingScore)
```

```
print(paste("Mean Income =", mean_income))
```

```
print(paste("Mean Spending Score =", mean_spending))
```

```
# =====
```

```
# Identify High Spending Customers
```

```
# Spending Score > 70
```

```
# =====
```

```
high_spending <- customer_df[  
  customer_df$SpendingScore > 70, ]
```

```
print("High Spending Customers")
```

```
print(high_spending)
```

```
# =====
```

```
# Display Structure and Summary
```

```
# =====
```

```
str(customer_df)
```

```
summary(customer_df)
```

```
# =====
```

```
# QUESTION 2
```

```
# DATA PREPROCESSING
```

```
# =====
```

```
# Create Dataset Directly
```

```
df <- data.frame(
```

```
  Age = c(22,25,30,35,40,28,32,45),
```

```
  Income = c(25000,30000,45000,50000,  
             65000,40000,48000,70000),
```

```
  SpendingScore = c(80,75,60,55,  
                    40,70,65,35),
```

```
  Gender = c("Male","Female","Female",  
             "Male","Male","Female",  
             "Female","Male")
```

```
)
```

```
print(df)
```

```
# =====
```

```
# Handle Missing Values
```

```
# =====
```

```
print(is.na(df))
```

```
# Replace missing income values
```

```
df$Income[is.na(df$Income)] <- mean(df$Income,  
                                     na.rm = TRUE)
```

```
print(df)
```

```
# =====
```

```
# Handle Invalid Values
```

```
# Remove negative income values
```

```
# =====
```

```
df <- df[df$Income > 0, ]
```

```
print(df)
```

```
# =====
```

```
# Normalize Variables
```

```
# =====
```

```
df$Income_Normalized <- scale(df$Income)
```

```
df$Spending_Normalized <- scale(df$SpendingScore)
```

```
print(df)
```

```
# =====
```

```
# Statistical Analysis
```

```
# =====
```

```
summary(df)
```

```
mean_income <- mean(df$Income)
```

```
median_income <- median(df$Income)
```

```
sd_income <- sd(df$Income)
```

```
variance_income <- var(df$Income)
```

```
print(paste("Mean Income =", mean_income))
```

```
print(paste("Median Income =", median_income))
```

```
print(paste("Standard Deviation =", sd_income))
```

```
print(paste("Variance =", variance_income))
```

```
# =====
```

```
# Scatter Plot Visualization
```

```
# =====
```

```
# NOTE:
```

```
# Commented because online compilers
```

```
# may not support graphics
```

```
# plot(df$Income,
```

```
#   df$SpendingScore,
```

```
#   main = "Income vs Spending Score",
```

```
#   xlab = "Income",
```

```
#   ylab = "Spending Score")
```

```
# =====
```

```
# Statistical Test
```

```
# Check whether income differs across age groups
```

```
# =====
```

```
# Create age groups
```

```
df$AgeGroup <- ifelse(df$Age < 30,  
  "Young",  
  "Adult")
```

```
# ANOVA Test
```

```
anova_result <- aov(Income ~ AgeGroup,  
  data = df)
```

```
summary(anova_result)
```

```
# =====
```

```
# QUESTION 3
```

```
# MACHINE LEARNING MODEL
```

```
# =====
```

```
# =====
```

```
# Apply K-Means Clustering
```

```
# =====
```

```
# Select numerical columns
```

```
cluster_data <- df[, c("Income",
```

```
"SpendingScore"]]
```

```
# Apply K-Means
```

```
set.seed(100)
```

```
kmeans_model <- kmeans(cluster_data,  
                        centers = 3)
```

```
print(kmeans_model)
```

```
# =====
```

```
# Determine Optimal Clusters
```

```
# =====
```

```
print("Number of Clusters = 3")
```

```
# =====
```

```
# Evaluate Clustering
```

```
# =====
```

```
print("Cluster Centers")
```

```
print(kmeans_model$centers)
```

```
print("Cluster Assignment")
```

```
print(kmeans_model$cluster)
```

```
# =====
```

```
# Add Cluster Labels to Dataset
```

```
# =====
```

```
df$Cluster <- kmeans_model$cluster
```

```
print(df)
```

```
# =====
```

```
# Visualize Clusters
```

```
# =====
```

```
# NOTE:
```

```
# Commented because online compilers
```

```
# may not support graphics
```

```
# plot(df$Income,
```

```
#   df$SpendingScore,
```

```
#   col = df$Cluster,
```

```
#   pch = 19,
```



```
# main = "Customer Clusters",  
# xlab = "Income",  
# ylab = "Spending Score")
```

```
# =====
```

```
# END OF PROGRAM
```

```
# =====
```

```
# =====
```

```
# SAC PRACTICAL - HOSPITAL DATASET
```

```
# PATIENT INFORMATION
```

```
# =====
```

```
# =====
```

```
# QUESTION 4
```

```
# Create list containing age, BP and cholesterol
```

```
# =====
```

```
patient_list <- list(
```

```
  Age = c(25,30,35,40,45,50,55,60),
```

```
  BP = c(120,130,125,140,150,135,145,160),
```

```
Cholesterol = c(180,190,200,210,  
                220,195,205,230)  
)
```

```
# Display list  
print(patient_list)
```

```
# =====  
# Convert List into Data Frame  
# =====
```

```
patient_df <- as.data.frame(patient_list)  
  
print(patient_df)
```

```
# =====  
# Compute Average Values  
# =====
```

```
avg_age <- mean(patient_df$Age)
```

```
avg_bp <- mean(patient_df$BP)
```

```
avg_cholesterol <- mean(patient_df$Cholesterol)
```

```
print(paste("Average Age =", avg_age))
```

```
print(paste("Average BP =", avg_bp))
```

```
print(paste("Average Cholesterol =",  
            avg_cholesterol))
```

```
# =====
```

```
# Identify High Risk Patients
```

```
# BP > 140 OR Cholesterol > 220
```

```
# =====
```

```
high_risk <- patient_df[  
  patient_df$BP > 140 |  
  patient_df$Cholesterol > 220, ]
```

```
print("High Risk Patients")
```

```
print(high_risk)
```

```
# =====
```

```
# Display Structure of Data Frame
```

```
# =====
```

```
str(patient_df)
```

```
# =====
```

```
# QUESTION 5
```

```
# DATA PREPROCESSING
```

```
# =====
```

```
# Create Medical Dataset Directly
```

```
df <- data.frame(
```

```
  Age = c(25,30,35,40,45,50,55,60),
```

```
  BP = c(120,130,125,140,150,135,145,160),
```

```
  Cholesterol = c(180,190,200,210,  
                  220,195,205,230),
```

```
  Group = c("A","A","A","B",  
            "B","B","A","B")
```

```
)
```

```
print(df)
```

```
# =====  
# Handle Missing Values  
# =====
```

```
print(is.na(df))
```

```
# Replace missing BP values  
df$BP[is.na(df$BP)] <- mean(df$BP,  
                             na.rm = TRUE)
```

```
print(df)
```

```
# =====  
# Handle Invalid Values  
# Remove negative values  
# =====
```

```
df <- df[df$BP > 0 &  
         df$Cholesterol > 0, ]
```

```
print(df)
```

```
# =====
```

```
# Normalize Data
```

```
# =====
```

```
df$BP_Normalized <- scale(df$BP)
```

```
df$Cholesterol_Normalized <-  
  scale(df$Cholesterol)
```

```
print(df)
```

```
# =====
```

```
# Statistical Analysis
```

```
# =====
```

```
summary(df)
```

```
mean_bp <- mean(df$BP)
```

```
median_bp <- median(df$BP)
```

```
sd_bp <- sd(df$BP)
```

```
variance_bp <- var(df$BP)
```

```
print(paste("Mean BP =", mean_bp))
```

```
print(paste("Median BP =", median_bp))
```

```
print(paste("Standard Deviation =", sd_bp))
```

```
print(paste("Variance =", variance_bp))
```

```
# =====
```

```
# Visualize Data
```

```
# =====
```

```
# NOTE:
```

```
# Commented because online compilers
```

```
# may not support graphics
```

```
# hist(df$BP,
```

```
#   main = "Blood Pressure Distribution",
```

```
#   xlab = "Blood Pressure")
```

```
# =====
```

```
# Statistical Test
```

```
# Compare BP between two groups
```

```
# =====
```

```
groupA_bp <- df$BP[df$Group == "A"]
```

```
groupB_bp <- df$BP[df$Group == "B"]
```

```
test_result <- t.test(groupA_bp,  
                      groupB_bp)
```

```
print(test_result)
```

```
# =====
```

```
# QUESTION 6
```

```
# MACHINE LEARNING MODEL
```

```
# =====
```

```
# =====
```

```
# Create Risk Column
```

```
# High if BP >= 140 else Low
```

```
# =====
```

```
df$Risk <- ifelse(df$BP >= 140,  
                 "High",  
                 "Low")
```

```
print(df)
```



```
# =====  
# Split Dataset into Training and Testing  
# =====  
  
set.seed(100)  
  
index <- sample(1:nrow(df),  
               0.7 * nrow(df))  
  
train <- df[index, ]  
  
test <- df[-index, ]  
  
print(train)  
  
print(test)  
  
  
# =====  
# Apply KNN Algorithm  
# =====  
  
# Install package if needed  
# install.packages("class")
```

```
library(class)
```

```
# =====
```

```
# Prepare Training and Testing Data
```

```
# =====
```

```
train_data <- train[, c("BP",  
                        "Cholesterol")]
```

```
test_data <- test[, c("BP",  
                     "Cholesterol")]
```

```
train_labels <- train$Risk
```

```
# =====
```

```
# Apply KNN with K = 3
```

```
# =====
```

```
pred <- knn(train = train_data,  
            test = test_data,  
            cl = train_labels,  
            k = 3)
```

```
print(pred)
```

```
# =====
```

```
# Evaluate Performance
```

```
# =====
```

```
confusion_matrix <- table(pred,  
                           test$Risk)
```

```
print(confusion_matrix)
```

```
# =====
```

```
# Accuracy Calculation
```

```
# =====
```

```
accuracy <- mean(pred == test$Risk)
```

```
print(paste("Accuracy =", accuracy))
```

```
# =====
```

```
# Try Different K Values
```

```
# =====
```

```
pred_k1 <- knn(train_data,
               test_data,
               cl = train_labels,
               k = 1)
```

```
pred_k5 <- knn(train_data,
               test_data,
               cl = train_labels,
               k = 5)
```

```
print("Prediction with K = 1")
print(pred_k1)
```

```
print("Prediction with K = 5")
print(pred_k5)
```

```
# =====
```

```
# Visualize Results
```

```
# =====
```

```
# NOTE:
```

```
# Commented because online compilers
```

```
# may not support graphics
```

```
# plot(df$BP,
```

```
# df$Cholesterol,  
# col = ifelse(df$Risk == "High",  
#             "red",  
#             "green"),  
# pch = 19,  
# main = "Patient Risk Levels",  
# xlab = "Blood Pressure",  
# ylab = "Cholesterol")
```

```
# =====
```

```
# END OF PROGRAM
```

```
# =====
```

```
# =====
```

```
# SAC PRACTICAL - EMAIL DATASET
```

```
# EMAIL SPAM DETECTION
```

```
# =====
```

```
# =====
```

```
# QUESTION 7
```

```
# Create 3D Array representing email features
```

```
# =====
```

```
email_array <- array(
```

```
c(
```

```
  # Email Length
```

```
  120,150,180,200,
```

```
  # Number of Links
```

```
  2,5,1,8,
```

```
  # Spam Score
```

```
  30,80,20,90
```

```
),
```

```
dim = c(2,2,3),
```

```
dimnames = list(
```

```
  Email = c("Email1", "Email2"),
```

```
  Feature = c("Feature1", "Feature2"),
```

```
  Metric = c("Length",
```

```
            "Links",
```

```
            "SpamScore")
```

```
)
```

```
)
```

```
# Display Array
```

```
print(email_array)
```

```
# =====
```

```
# Perform Summary Operations
```

```
# =====
```

```
total_values <- sum(email_array)
```

```
average_values <- mean(email_array)
```

```
max_value <- max(email_array)
```

```
min_value <- min(email_array)
```

```
print(paste("Total =", total_values))
```

```
print(paste("Average =", average_values))
```

```
print(paste("Maximum Value =", max_value))
```

```
print(paste("Minimum Value =", min_value))
```

```
# =====
```

```
# Identify Patterns in Data
```

```
# =====
```

```
print("Higher Spam Score may indicate spam emails")
```

```
print(email_array)
```

```
# =====
```

```
# Interpret Results
```

```
# =====
```

```
print("Emails with more links and higher spam score")
```

```
print("are more likely to be spam emails")
```

```
# =====
```

```
# Convert Array into Data Frame
```

```
# =====
```

```
email_df <- as.data.frame.table(email_array)
```

```
print(email_df)
```

```
str(email_df)
```



```

# =====

# QUESTION 8

# DATA PREPROCESSING

# =====


# Create Email Dataset Directly


df <- data.frame(

  EmailLength = c(120,150,180,200,
                  140,160,210,190),

  Links = c(2,5,1,8,3,6,9,4),

  SpamScore = c(30,80,20,90,
                40,75,95,50),

  Type = c("NotSpam","Spam",
            "NotSpam","Spam",
            "NotSpam","Spam",
            "Spam","NotSpam")
)

print(df)

```

```
# =====
```

```
# Handle Missing Values
```

```
# =====
```

```
print(is.na(df))
```

```
# Replace missing values
```

```
df$SpamScore[is.na(df$SpamScore)] <-
```

```
  mean(df$SpamScore,
```

```
    na.rm = TRUE)
```

```
print(df)
```

```
# =====
```

```
# Handle Invalid Values
```

```
# Remove negative values
```

```
# =====
```

```
df <- df[df$SpamScore >= 0, ]
```

```
print(df)
```

```
# =====
```

```
# Normalize Features
```

```
# =====
```

```
df$SpamScore_Normalized <-
```

```
  scale(df$SpamScore)
```

```
df$Links_Normalized <-
```

```
  scale(df$Links)
```

```
print(df)
```

```
# =====
```

```
# Statistical Analysis
```

```
# =====
```

```
summary(df)
```

```
mean_spam <- mean(df$SpamScore)
```

```
median_spam <- median(df$SpamScore)
```

```
sd_spam <- sd(df$SpamScore)
```

```
variance_spam <- var(df$SpamScore)
```

```
print(paste("Mean Spam Score =",
```

```

    mean_spam))

print(paste("Median Spam Score =",
    median_spam))

print(paste("Standard Deviation =",
    sd_spam))

print(paste("Variance =",
    variance_spam))


# =====
# Visualize Distributions
# =====

# NOTE:
# Commented because online compilers
# may not support graphics

# hist(df$SpamScore,
#   main = "Spam Score Distribution",
#   xlab = "Spam Score")

# =====

```

```

# Statistical Test

# Check association between email type
# and categorical feature
# =====

# Create categorical feature
df$LinkCategory <- ifelse(df$Links > 4,
                          "HighLinks",
                          "LowLinks")

# Create contingency table
table_data <- table(df$Type,
                    df$LinkCategory)

print(table_data)

# Fisher Exact Test
fisher_result <- fisher.test(table_data)

print(fisher_result)

# =====

# QUESTION 9

# MACHINE LEARNING MODEL

# =====

```

```
# =====  
# Split Dataset into Training and Testing  
# =====
```

```
set.seed(100)
```

```
index <- sample(1:nrow(df),  
               0.7 * nrow(df))
```

```
train <- df[index, ]
```

```
test <- df[-index, ]
```

```
print(train)
```

```
print(test)
```

```
# =====  
# Apply Naive Bayes Classifier  
# =====
```

```
# Install package if needed  
# install.packages("e1071")
```

```
library(e1071)
```

```
# Build Model
```

```
model <- naiveBayes(Type ~ EmailLength +  
  Links +  
  SpamScore,  
  data = train)
```

```
print(model)
```

```
# =====
```

```
# Prediction
```

```
# =====
```

```
pred <- predict(model,  
  test)
```

```
print(pred)
```

```
# =====
```

```
# Confusion Matrix
```

```
# =====
```

```
confusion_matrix <- table(pred,
```

```
test$Type)
```

```
print(confusion_matrix)
```

```
# =====
```

```
# Accuracy Calculation
```

```
# =====
```

```
accuracy <- mean(pred == test$Type)
```

```
print(paste("Accuracy =", accuracy))
```

```
# =====
```

```
# Interpret Probabilities
```

```
# =====
```

```
probabilities <- predict(model,
```

```
    test,
```

```
    type = "raw")
```

```
print(probabilities)
```



```
# =====  
  
# Visualize Results  
  
# =====  
  
# NOTE:  
  
# Commented because online compilers  
# may not support graphics  
  
# plot(df$Links,  
#      df$SpamScore,  
#      col = ifelse(df$Type == "Spam",  
#                   "red",  
#                   "green"),  
#      pch = 19,  
#      main = "Spam vs Not Spam",  
#      xlab = "Links",  
#      ylab = "Spam Score")  
  
# =====  
  
# END OF PROGRAM  
  
# =====  
  
# =====  
  
# SAC PRACTICAL - BILLING DATASET  
# SHOP BILLING RECORDS  
  
# =====
```

```

# =====

# QUESTION 10

# Create vector storing bill amounts

# =====

bill_amounts <- c(450,520,610,480,750,680,
                  590,430,800,720,560,490)

# Display bill amounts
print(bill_amounts)


# =====

# Compute Total Revenue and Average Bill

# =====

total_revenue <- sum(bill_amounts)

average_bill <- mean(bill_amounts)

print(paste("Total Revenue =", total_revenue))

print(paste("Average Bill =", average_bill))

```

```
# =====  
# Identify Highest and Lowest Bill Amounts  
# =====
```

```
highest_bill <- max(bill_amounts)
```

```
lowest_bill <- min(bill_amounts)
```

```
print(paste("Highest Bill Amount =",  
            highest_bill))
```

```
print(paste("Lowest Bill Amount =",  
            lowest_bill))
```

```
# =====  
# Compare First 6 and Last 6 Bills  
# =====
```

```
avg_first6 <- mean(bill_amounts[1:6])
```

```
avg_last6 <- mean(bill_amounts[7:12])
```

```
print(paste("Average of First 6 Bills =",  
            avg_first6))
```

```
print(paste("Average of Last 6 Bills =",  
          avg_last6))
```

```
# =====
```

```
# Convert Vector into Data Frame
```

```
# =====
```

```
billing_df <- data.frame(BillAmount =  
                        bill_amounts)
```

```
print(billing_df)
```

```
str(billing_df)
```

```
# =====
```

```
# QUESTION 11
```

```
# DATA PREPROCESSING
```

```
# =====
```

```
# Create Billing Dataset Directly
```

```
df <- data.frame(
```

```

BillAmount = c(450,520,610,480,
               750,680,590,430,
               800,720,560,490)
)

print(df)

# =====
# Handle Missing Values
# =====

print(is.na(df))

# Replace missing values with mean
df$BillAmount[is.na(df$BillAmount)] <-
  mean(df$BillAmount,
       na.rm = TRUE)

print(df)

# =====
# Handle Outliers
# Remove bills greater than 1000
# =====

```

```
df <- df[df$BillAmount <= 1000, ]
```

```
print(df)
```

```
# =====
```

```
# Normalize Bill Amounts
```

```
# =====
```

```
df$Normalized_Bill <-
```

```
  scale(df$BillAmount)
```

```
print(df)
```

```
# =====
```

```
# Statistical Analysis
```

```
# =====
```

```
summary(df)
```

```
mean_bill <- mean(df$BillAmount)
```

```
median_bill <- median(df$BillAmount)
```

```
sd_bill <- sd(df$BillAmount)
```

```
variance_bill <- var(df$BillAmount)
```

```
print(paste("Mean Bill =", mean_bill))
```

```
print(paste("Median Bill =", median_bill))
```

```
print(paste("Standard Deviation =", sd_bill))
```

```
print(paste("Variance =", variance_bill))
```

```
# =====
```

```
# Visualize Bill Distribution
```

```
# =====
```

```
# NOTE:
```

```
# Commented because online compilers
```

```
# may not support graphics
```

```
# hist(df$BillAmount,
```

```
#   main = "Bill Distribution",
```

```
#   xlab = "Bill Amount")
```

```
# =====
```

```
# Statistical Test
```

```
# Check whether average bill differs from 500
```

```
# =====
```

```
test_result <- t.test(df$BillAmount,  
                      mu = 500)
```

```
print(test_result)
```

```
# =====
```

```
# QUESTION 12
```

```
# MACHINE LEARNING MODEL
```

```
# =====
```

```
# =====
```

```
# Create Additional Feature
```

```
# =====
```

```
df$CustomerVisits <- c(2,3,4,2,  
                       5,4,3,2,  
                       6,5,3,2)
```

```
print(df)
```



```
# =====  
# Split Dataset into Training and Testing  
# =====  
  
set.seed(100)  
  
index <- sample(1:nrow(df),  
               0.7 * nrow(df))  
  
train <- df[index, ]  
  
test <- df[-index, ]  
  
print(train)  
  
print(test)  
  
  
# =====  
# Apply Linear Regression  
# Predict BillAmount using CustomerVisits  
# =====  
  
model <- lm(BillAmount ~ CustomerVisits,
```

```

data = train)

print(summary(model))

# =====

# Prediction

# =====

pred <- predict(model,
                 test)

print(pred)

# =====

# RMSE Calculation

# =====

rmse <- sqrt(mean((pred -
                  test$BillAmount)^2))

print(paste("RMSE =", rmse))

```

```
# =====  
  
# Visualize and Interpret Results  
  
# =====  
  
# NOTE:  
  
# Commented because online compilers  
# may not support graphics  
  
# plot(test$CustomerVisits,  
#       test$BillAmount,  
#       main = "Actual Bill Amount",  
#       xlab = "Customer Visits",  
#       ylab = "Bill Amount")  
  
# =====  
  
# END OF PROGRAM  
  
# =====  
  
# =====  
  
# SAC PRACTICAL - SALES DATASET  
# COMPANY MONTHLY SALES  
  
# =====
```

```
# =====
```

```
# QUESTION 13
```

```
# Create Matrix (Products x Months)
```

```
# =====
```

```
sales_matrix <- matrix(c(
```

```
  # January Sales
```

```
  500, 700, 650,
```

```
  # February Sales
```

```
  550, 720, 680,
```

```
  # March Sales
```

```
  600, 750, 700,
```

```
  # April Sales
```

```
  620, 770, 730
```

```
),
```

```
  nrow = 3,
```

```
  ncol = 4,
```

```
  byrow = FALSE)
```

```
# Row Names
```

```
rownames(sales_matrix) <- c("ProductA",
```

```
  "ProductB",
```

```
"ProductC")
```

```
# Column Names
```

```
colnames(sales_matrix) <- c("Jan",  
                             "Feb",  
                             "Mar",  
                             "Apr")
```

```
# Display Matrix
```

```
print(sales_matrix)
```

```
# =====
```

```
# Compute Row-wise Totals
```

```
# =====
```

```
row_totals <- rowSums(sales_matrix)
```

```
print("Row Wise Totals")
```

```
print(row_totals)
```

```
# =====
```

```
# Compute Column-wise Totals
```

```
# =====
```

```
column_totals <- colSums(sales_matrix)
```

```
print("Column Wise Totals")
```

```
print(column_totals)
```

```
# =====
```

```
# Identify Highest Selling Product
```

```
# =====
```

```
highest_product <- names(which.max(row_totals))
```

```
highest_sales <- max(row_totals)
```

```
print(paste("Highest Selling Product =",  
            highest_product))
```

```
print(paste("Highest Sales =",  
            highest_sales))
```

```
# =====
```

```
# Compare Monthly Trends
```

```
# =====
```

```
print("Monthly Sales Trends")
```

```
print(sales_matrix)
```

```
# =====
```

```
# Convert Matrix into Data Frame
```

```
# =====
```

```
sales_df <- as.data.frame(sales_matrix)
```

```
print(sales_df)
```

```
str(sales_df)
```

```
# =====
```

```
# QUESTION 14
```

```
# DATA PREPROCESSING
```

```
# =====
```

```
# Create Dataset Directly
```

```
df <- data.frame(
```

```
Product = c("A","B","C","D",  
            "E","F","G","H"),
```

```
Jan = c(500,700,650,720,  
        680,750,800,620),
```

```
Feb = c(550,720,680,740,  
        700,780,820,640),
```

```
Mar = c(600,750,700,760,  
        730,800,850,670),
```

```
Apr = c(620,770,730,780,  
        760,830,880,690)
```

```
)
```

```
print(df)
```

```
# =====
```

```
# Handle Missing Values
```

```
# =====
```

```
print(is.na(df))
```

```
# Replace missing values with column mean
```



```
df$Jan[is.na(df$Jan)] <-  
  mean(df$Jan, na.rm = TRUE)
```

```
df$Feb[is.na(df$Feb)] <-  
  mean(df$Feb, na.rm = TRUE)
```

```
df$Mar[is.na(df$Mar)] <-  
  mean(df$Mar, na.rm = TRUE)
```

```
df$Apr[is.na(df$Apr)] <-  
  mean(df$Apr, na.rm = TRUE)
```

```
print(df)
```

```
# =====
```

```
# Normalize Data
```

```
# =====
```

```
df$Jan_Normalized <- scale(df$Jan)
```

```
df$Feb_Normalized <- scale(df$Feb)
```

```
print(df)
```

```
# =====  
  
# Statistical Analysis  
  
# =====  
  
summary(df)  
  
mean_sales <- mean(df$Jan)  
  
median_sales <- median(df$Jan)  
  
sd_sales <- sd(df$Jan)  
  
variance_sales <- var(df$Jan)  
  
print(paste("Mean Sales =", mean_sales))  
  
print(paste("Median Sales =", median_sales))  
  
print(paste("Standard Deviation =", sd_sales))  
  
print(paste("Variance =", variance_sales))  
  
  
  
  
  
  
  
  
  
# =====  
  
# Visualize Sales Trends  
  
# =====
```

```
# NOTE:
```

```
# Commented because online compilers
```

```
# may not support graphics
```

```
# plot(df$Jan,
```

```
#   type = "o",
```

```
#   main = "January Sales Trend",
```

```
#   xlab = "Products",
```

```
#   ylab = "Sales")
```

```
# =====
```

```
# Statistical Test
```

```
# Check whether sales differ across months
```

```
# =====
```

```
sales_values <- c(df$Jan,
```

```
                df$Feb,
```

```
                df$Mar,
```

```
                df$Apr)
```

```
month_group <- factor(c(
```

```
  rep("Jan", nrow(df)),
```

```
  rep("Feb", nrow(df)),
```

```
  rep("Mar", nrow(df)),
```

```
  rep("Apr", nrow(df))
```

```
))
```

```
anova_result <- aov(sales_values ~ month_group)
```

```
summary(anova_result)
```

```
# =====
```

```
# QUESTION 15
```

```
# MACHINE LEARNING MODEL
```

```
# =====
```

```
# =====
```

```
# Prepare Data for Clustering
```

```
# =====
```

```
cluster_data <- df[, c("Jan",  
                      "Feb",  
                      "Mar",  
                      "Apr")]
```

```
print(cluster_data)
```

```
# =====
```

```
# Apply K-Means Clustering
```

```
# =====
```

```
set.seed(100)
```

```
kmeans_model <- kmeans(cluster_data,  
                        centers = 3)
```

```
print(kmeans_model)
```

```
# =====
```

```
# Determine Optimal Clusters
```

```
# =====
```

```
print("Number of Clusters = 3")
```

```
# =====
```

```
# Evaluate Clusters
```

```
# =====
```

```
print("Cluster Centers")
```

```
print(kmeans_model$centers)
```

```
print("Cluster Assignments")
```

```
print(kmeans_model$cluster)
```

```
# =====
```

```
# Add Cluster Labels to Dataset
```

```
# =====
```

```
df$Cluster <- kmeans_model$cluster
```

```
print(df)
```

```
# =====
```

```
# Visualize Clusters
```

```
# =====
```

```
# NOTE:
```

```
# Commented because online compilers
```

```
# may not support graphics
```

```
# plot(df$Jan,
```

```
#   df$Feb,
```

```
# col = df$Cluster,  
# pch = 19,  
# main = "Sales Clusters",  
# xlab = "January Sales",  
# ylab = "February Sales")
```

```
# =====
```

```
# END OF PROGRAM
```

```
# =====
```